

DevOps Task Manager: Comprehensive Project Report

Project Overview

The **DevOps Task Manager** is a modern, responsive web application designed to demonstrate a fully automated, end-to-end DevOps lifecycle. This project showcases the integration of multiple enterprise-grade tools to achieve Infrastructure as Code (IaC), containerization, continuous integration/continuous deployment (CI/CD), orchestration, and cluster observability.

Unlike traditional static deployments, this project utilizes a dynamic architecture where the Jenkins pipeline automatically provisions the AWS infrastructure from scratch, builds the application, deploys it to a self-hosted Kubernetes cluster, and offers an automated teardown process to optimize cloud costs.

Technologies Used

- **Frontend:** React (Vite), Vanilla CSS (Glassmorphism design)
- **Version Control:** Git & GitHub
- **CI/CD:** Jenkins (Windows Agent)
- **Containerization:** Docker & DockerHub
- **Infrastructure as Code (IaC):** Terraform, AWS CLI
- **Cloud Provider:** Amazon Web Services (AWS EC2)
- **Container Orchestration:** Kubernetes (k3s distribution)
- **Monitoring & Observability:** Prometheus, Grafana

Project Architecture

The architecture is designed to be lightweight yet highly scalable. An AWS EC2 instance serves as the foundation, running a single-node k3s Kubernetes cluster. The application traffic is routed through a Kubernetes NodePort Service.

```
graph TD
    Client((End User)) -->|HTTP:30080| EC2[AWS EC2 Instance]

    subgraph "Kubernetes (k3s) Cluster"
        EC2 --> Nginx[Nginx Ingress / Service]
        Nginx --> Pod1[Task Manager Pod 1]
        Nginx --> Pod2[Task Manager Pod 2]
    end

    subgraph "Monitoring Stack"
        Prometheus[Prometheus Server]
        Grafana[Grafana Dashboard]
    end

    end
```

CI/CD Pipeline Flow

The Jenkins pipeline is the heart of this project, fully automating the transition from code commit to live deployment.

```
sequenceDiagram
    participant Dev as Developer
    participant Git as GitHub
    participant J as Jenkins
    participant TF as Terraform
    participant DH as DockerHub
    participant AWS as AWS EC2 / k3s

    Dev->>Git: 1. Push code changes
    Git->>J: 2. Trigger Pipeline
    J->>TF: 3. terraform apply
    TF->>AWS: 4. Provision EC2 & install k3s
    J->>J: 5. docker build
    J->>DH: 6. docker push (komalpreet1703/task-manager)
    J->>AWS: 7. SCP manifests via SSH
    J->>AWS: 8. SSH: kubect1 apply
    AWS->>DH: 9. Pull latest image
    J->>TF: 10. (Optional) terraform destroy
```

Folder Structure

The repository is modularly structured, separating the frontend application from the DevOps configurations:

```
devops-task-manager/
├── app/                # React source code (App.jsx, index.css, Vite config)
├── docker/             # Dockerfile (Multi-stage build)
├── jenkins/            # Declarative Jenkinsfile (CI/CD definitions)
├── k8s/                # Kubernetes manifests (deployment.yaml, service.yaml)
├── monitoring/         # Prometheus and Grafana manifests
├── terraform/          # Infrastructure as Code (main.tf, variables.tf)
├── .gitignore          # Git exclusions (Node, Terraform)
└── README.md           # Standard repository documentation
```

Prerequisites and Setup

To replicate this environment, the following prerequisites must be met:

1. **AWS Account:** An active AWS account with the AWS CLI configured locally (`aws configure`).
2. **Jenkins Server:** A running Jenkins instance with the Pipeline and SSH plugins installed.
3. **DockerHub Account:** A public repository to host the Docker images.
4. **Jenkins Credentials:**
 - `dockerhub-creds` (Username with password)
 - `aws-ssh-key` (SSH Username with private key `.pem` file)

Docker Configuration

The application is containerized using a highly optimized **Multi-Stage Dockerfile**:

- **Stage 1 (Build):** Uses `node:20-alpine` to install dependencies and compile the Vite React application into static files.
 - **Stage 2 (Serve):** Uses a lightweight `nginx:alpine` image to serve the compiled static files on port 80.
- This approach reduces the final image size significantly and eliminates security vulnerabilities associated with shipping development dependencies to production.

Terraform Infrastructure

Terraform automates the AWS infrastructure provisioning, ensuring the environment is reproducible and idempotent.

- **Provider:** AWS (`us-east-1`).
- **Resources:**
 - `aws_security_group`: Opens ports 22 (SSH), 80 (HTTP), 6443 (K8s API), and the 30000-32767 (NodePort) range.
 - `aws_instance`: Provisions a `t3.micro` EC2 instance.
- **Bootstrapping (User Data):** A bash script embedded in Terraform automatically installs Docker, configures k3s, and adjusts permissions for the `ubuntu` user during the EC2 boot sequence.

Jenkins Pipeline

The `Jenkinsfile` is written in Declarative Pipeline syntax, customized for a Windows Jenkins Agent using native `bat` and `powershell` commands.

Key Pipeline Stages:

1. **Infrastructure Provisioning:** Executes `terraform apply` and extracts the dynamic AWS Public IP.
2. **Docker Build & Push:** Compiles the Docker image, tags it with the Jenkins Build Number, and pushes it to DockerHub.
3. **Remote SSH Deployment:**
 - Uses PowerShell to dynamically inject the new Docker tag into the `deployment.yaml`.
 - Executes a 3-minute sleep timer to allow Kubernetes to initialize on AWS.
 - Uses `scp` to transfer the manifest folders to the EC2 instance.
 - Uses `ssh` to execute `kubectl apply` remotely, eliminating the need to store dynamic `kubeconfig` files locally.
4. **Infrastructure Teardown:** Pauses for manual approval before running `terraform destroy` to optimize cloud billing.

Kubernetes (k3s) Deployment

The orchestration layer ensures the React application is highly available.

- **Deployment (`deployment.yaml`):** Maintains a ReplicaSet of exactly 2 Pods running the React application. It defines resource requests (CPU/Memory) to prevent node exhaustion.
- **Service (`service.yaml`):** Uses a `NodePort` service type to expose the internal Pods to the external internet on port 30080.

Monitoring with Prometheus & Grafana

Observability is built into the cluster using standard open-source tools:

- **Prometheus (`prometheus.yaml`):** Deployed alongside a ConfigMap to scrape metrics from the Kubernetes nodes and pods every 15 seconds. Exposed on port 30090.
- **Grafana (`grafana.yaml`):** Provides a visual dashboard to interpret Prometheus data. Exposed on port 30030.

Verification Steps and URLs

Once the Jenkins pipeline completes the deployment stage, the applications are immediately accessible via the EC2 instance's dynamic Public IP:

- **Task Manager UI:** `http://<EC2_PUBLIC_IP>:30080`
- **Prometheus Target Status:** `http://<EC2_PUBLIC_IP>:30090/targets`
- **Grafana Dashboard:** `http://<EC2_PUBLIC_IP>:30030` (Default Login: `admin / admin123`)

Challenges Faced & Solutions

During the development of this project, several significant DevOps challenges were encountered and successfully resolved:

1. GitHub File Size Limitations (Git Tracking)

- **Challenge:** An attempt to push the repository to GitHub was rejected because Terraform downloaded a 685MB AWS provider executable inside the `.terraform/` directory, violating GitHub's 100MB file limit.
- **Solution:** Implemented a comprehensive `.gitignore` file and used `git rm -r --cached` to wipe the massive executables and sensitive `.tfstate` files from Git's tracking history, allowing for a successful push.

2. Strict AWS Free-Tier Constraints

- **Challenge:** The `terraform apply` command failed during EC2 provisioning because the AWS account strictly enforced free-tier resource limits, rejecting the initially configured `t2.medium` instance.
- **Solution:** Diagnosed the account's specific free-tier eligibility using the AWS CLI (`aws ec2 describe-instance-types`) and successfully refactored the Terraform variables to use `t3.micro`.

3. Dynamic Pipeline Architecture (The "Chicken and Egg" Problem)

- **Challenge:** Because Terraform provisions the EC2 instance *inside* the Jenkins pipeline, the Kubernetes `kubeconfig` was ephemeral. Jenkins could not use a pre-saved Kubernetes credential to deploy the application because the cluster didn't exist when the pipeline started.
- **Solution:** Refactored the Jenkins pipeline to utilize a "Remote SSH" deployment pattern. Instead of Jenkins communicating with the Kubernetes API, Jenkins securely copies (`scp`) the manifests to the new server and executes `kubectl` locally on the server via `ssh`.

4. Windows Jenkins Agent Compatibility

- **Challenge:** The Jenkins pipeline failed to execute standard Linux `sh` and `sed` commands because the Jenkins agent was running natively on a Windows OS.
- **Solution:** Migrated all shell execution steps to Windows `bat` commands and replaced Linux string-manipulation tools with native PowerShell commands (`powershell -Command "(Get-Content ...) -replace ..."`) for cross-platform compatibility.

Conclusion

The DevOps Task Manager project successfully bridges the gap between software development and IT operations. By treating infrastructure as code, containerizing the application, and automating the entire lifecycle through Jenkins, the project demonstrates a highly scalable, fault-tolerant, and professional DevOps architecture. The ability to provision, deploy, and monitor an application entirely hands-off proves the immense value and efficiency of modern CI/CD practices.